**TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG**
**KHOA CÔNG NGHỆ THÔNG TIN**

**BÁO CÁO GIỮA KÌ**

# HỌC PHẦN CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

**ĐỀ TÀI:**

# QUẢN LÝ SẢN PHẨM BẰNG CÂY TÌM KIẾM NHỊ PHÂN

Người hướng dẫn : ***Nguyễn Thị Xuân Thủy***
Sinh viên thực hiện : ***Nguyễn Như Hùng***
***Phạm Anh Tú***
Lớp : ***25CT1***

**Đà Nẵng, 10/2026**

# Nhận Xét Của Người Hướng Dẫn

# TÓM TẮT

**Tên đề tài:** Quản lý sản phẩm bằng cây tìm kiếm nhị phân

**Sinh viên thực hiện:** Nguyễn Như Hùng | Phạm Anh Tú.

- Đề tài "Quản lý sản phẩm bằng cây tìm kiếm nhị phân" được thực hiện trong khuôn khổ học phần Cấu trúc dữ liệu và giải thuật, nhằm vận dụng kiến thức về cây nhị phân tìm kiếm vào việc xây dựng một chương trình quản lý thông tin sản phẩm bằng ngôn ngữ lập trình C++.
- Trong bài toán này, mã sản phẩm được sử dụng làm khóa để tổ chức và tìm kiếm dữ liệu. Mỗi sản phẩm bao gồm năm thuộc tính chính: mã sản phẩm, tên sản phẩm, đơn giá, số lượng và loại sản phẩm.
- Các nút được sắp xếp theo mã sản phẩm, bảo đảm khóa của các nút thuộc cây con trái nhỏ hơn khóa nút hiện tại và khóa của các nút thuộc cây con phải lớn hơn khóa nút hiện tại.
- Kết quả của đề tài là xây dựng được mô hình chương trình quản lý sản phẩm có cấu trúc rõ ràng, kết hợp kiến thức về cây nhị phân, đệ quy, con trỏ, xử lý tệp và thuật toán tìm kiếm. Tuy nhiên, chương trình vẫn còn một số hạn chế như chưa tự cân bằng cây, chưa kiểm tra đầy đủ dữ liệu đầu vào và chưa có giao diện đồ họa. Trong tương lai, hệ thống có thể được cải tiến bằng cách sử dụng cây AVL hoặc Red-Black Tree, tăng cường xử lý lỗi, mở rộng chức năng tìm kiếm và thống kê, đồng thời nâng cấp phương thức lưu trữ dữ liệu.

# LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn chân thành và sâu sắc đến **cô Nguyễn Thị Xuân Thủy**, giảng viên Khoa **Công Nghệ Thông Tin**, người đã tận tâm giảng dạy, truyền đạt những kiến thức quý báu và luôn tận tình chỉ bảo, định hướng cho em trong suốt quá trình học tập tại trường.

Em cũng xin gửi lời cảm ơn chân thành đến **các bạn học cùng lớp**, những người đã luôn đồng hành, chia sẻ kiến thức, kinh nghiệm và động viên em trong suốt thời gian thực hiện đề tài. Sự giúp đỡ và khích lệ của mọi người là nguồn động lực lớn giúp em vượt qua khó khăn, hoàn thành tốt quá trình nghiên cứu và phát triển sản phẩm.

Cuối cùng, em xin kính chúc quý thầy cô trong Khoa Công Nghệ Thông Tin luôn dồi dào sức khỏe, hạnh phúc và thành công trong sự nghiệp giảng dạy.

**Em xin chân thành cảm ơn!**

.

# CAM ĐOAN

Nhóm em xin cam đoan rằng **báo cáo giữa kì với đề tài "Quản lý sản phẩm bằng cây tìm kiếm nhị phân"** là **kết quả nghiên cứu và thực hiện riêng của nhóm** dưới sự hướng dẫn của **giảng viên hướng dẫn**.

Toàn bộ nội dung trong báo cáo được nhóm em tìm hiểu, tổng hợp và trình bày dựa trên kiến thức đã học, cùng với các tài liệu tham khảo có nguồn gốc rõ ràng và được trích dẫn đầy đủ theo quy định.

Nhóm em xin **chịu hoàn toàn trách nhiệm** về tính trung thực và liêm chính học thuật của báo cáo này. Mọi hành vi sao chép, gian lận hoặc vi phạm bản quyền (nếu có) em xin hoàn toàn chịu trách nhiệm.

**Sinh viên thực hiện**

Nguyễn Như Hùng | Phạm Anh Tú.

# MỤC LỤC

MỞ ĐẦU ..... 1
CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI ..... 1
1.1. Lý do chọn đề tài ..... 1
1.2. Mục tiêu đề tài ..... 1
1.3. Đối tượng và phạm vi ..... 1
1.4. Phương pháp thực hiện ..... 1
KẾT CHƯƠNG 1 ..... 2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT ..... 3
2.1. Khái niệm cây và cây nhị phân ..... 3
2.2. Cây tìm kiếm nhị phân (BST) ..... 3
2.3. Cấu trúc dữ liệu sử dụng ..... 3
2.4. Các phép duyệt cây ..... 4
2.5. Thuật toán thêm và tìm kiếm ..... 4
2.6. Thuật toán xóa nút ..... 4
2.7. Độ phức tạp thuật toán ..... 5
KẾT CHƯƠNG 2 ..... 6
CHƯƠNG 3. PHÂN TÍCH CHƯƠNG TRÌNH ..... 7
3.1. Bài toán và dữ liệu đầu vào/đầu ra ..... 7
3.2. Yêu cầu chức năng ..... 7
3.5. Sơ đồ luồng xử lý tổng quát ..... 8
3.6. Thiết kế menu ..... 9
3.7. Quy tắc và giả định dữ liệu ..... 9
KẾT CHƯƠNG 3 ..... 10
CHƯƠNG 4. CÀI ĐẶT CÁC CHỨC NĂNG ..... 11
4.1. Khởi tạo cây và tạo nút ..... 11
4.2. Nhập và in thông tin sản phẩm ..... 11
4.3. Thêm sản phẩm ..... 11
4.4. Tìm sản phẩm theo mã ..... 11
4.5. Xóa sản phẩm ..... 11
4.6. Cập nhật thông tin sản phẩm ..... 12

4.7. Duyệt NLR, LNR và LRN ..... 12
4.8. Tìm mã nhỏ nhất và lớn nhất ..... 13
4.9. Tìm sản phẩm có giá cao nhất ..... 13
4.10. Tìm sản phẩm có số lượng lớn nhất ..... 13
KẾT CHƯƠNG 4 ..... 14
KẾT LUẬN ..... 15

# MỞ ĐẦU

Trong các chương trình quản lý, dữ liệu cần được tổ chức sao cho việc thêm mới, tra cứu, cập nhật, xóa và thống kê có thể thực hiện thuận tiện. Nếu dữ liệu chỉ được lưu tuần tự trong một danh sách, việc tìm kiếm theo khóa có thể phải kiểm tra lần lượt nhiều phần tử. Vì vậy, lựa chọn cấu trúc dữ liệu phù hợp là một nội dung quan trọng trong thiết kế thuật toán.

Cây tìm kiếm nhị phân (Binary Search Tree – BST) là cấu trúc dữ liệu dạng cây có quy tắc sắp xếp khóa ở nhánh trái nhỏ hơn khóa của nút hiện tại và khóa ở nhánh phải lớn hơn khóa của nút hiện tại. Quy tắc này cho phép thực hiện tìm kiếm theo hướng đi xuống một nhánh thay vì luôn phải duyệt toàn bộ dữ liệu. Khi duyệt cây theo thứ tự trái – nút – phải, các khóa được xuất ra theo thứ tự tăng dần.

Đề tài **"Quản lý sản phẩm bằng cây tìm kiếm nhị phân"** sử dụng mã sản phẩm làm khóa. Mỗi nút chứa mã sản phẩm, tên sản phẩm, đơn giá, số lượng và loại sản phẩm. Chương trình được viết bằng C++, vận hành thông qua menu trên giao diện dòng lệnh.

# CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

## 1.1. Lý do chọn đề tài

Trong khuôn khổ học phần Cấu trúc dữ liệu và giải thuật, giảng viên đã đưa ra danh sách các đề tài để sinh viên lựa chọn và thực hiện nhằm củng cố kiến thức lý thuyết, đồng thời vận dụng các thuật toán và cấu trúc dữ liệu đã được học vào giải quyết những bài toán cụ thể. Dựa trên danh sách đề tài do giảng viên cung cấp, em đã lựa chọn Đề tài 10: Quản lý sản phẩm bằng **cây tìm kiếm nhị phân (Binary Search Tree – BST)** để nghiên cứu và triển khai.

Đề tài tập trung xây dựng chương trình quản lý thông tin sản phẩm bằng ngôn ngữ lập trình C++. Mỗi sản phẩm bao gồm các thông tin cơ bản như mã sản phẩm, tên sản phẩm, đơn giá, số lượng và loại sản phẩm. Trong đó, mã sản phẩm được sử dụng làm khóa để tổ chức dữ liệu trong cây tìm kiếm nhị phân, giúp thực hiện các thao tác quản lý và truy xuất thông tin theo yêu cầu.

## 1.2. Mục tiêu đề tài

Xây dựng chương trình quản lý sản phẩm bằng ngôn ngữ C++ theo mô hình cây tìm kiếm nhị phân. Sử dụng mã sản phẩm làm khóa và không cho phép hai sản phẩm có cùng mã trong cây.

Triển khai các thao tác thêm, tìm kiếm, xóa, cập nhật và duyệt cây theo ba thứ tự NLR, LNR, LRN. Tìm sản phẩm có mã nhỏ nhất và lớn nhất dựa trên đặc tính của BST.

## 1.3. Đối tượng và phạm vi

Đối tượng nghiên cứu là cấu trúc cây tìm kiếm nhị phân và các thuật toán thao tác trên cây. Đối tượng dữ liệu được quản lý là sản phẩm có năm thuộc tính: mã, tên, đơn giá, số lượng và loại.

Phạm vi triển khai hiện tại là ứng dụng console chạy cục bộ. Chương trình sử dụng thư viện chuẩn C++ gồm iostream, fstream, string, queue và limits; dữ liệu được lưu trong bộ nhớ động và có thể ghi ra tệp văn bản. Phiên bản này chưa có giao diện đồ họa, chưa kết nối cơ sở dữ liệu, chưa có phân quyền người dùng và chưa hỗ trợ nhiều người thao tác đồng thời.

## 1.4. Phương pháp thực hiện

Phân tích đề bài để xác định các thuộc tính sản phẩm và nhóm chức năng cần có. Thiết kế cấu trúc SanPham, Node và con trỏ gốc root. Xây dựng từng hàm xử lý cây và kết nối chúng với menu chính. Đối chiếu từng chức năng với mã nguồn thực tế để mô tả đúng cách hoạt động.

# KẾT CHƯƠNG 1

Trong chương 1, báo cáo đã trình bày tổng quan về đề tài Quản lý sản phẩm bằng cây tìm kiếm nhị phân (BST), bao gồm lý do lựa chọn đề tài, mục tiêu thực hiện, đối tượng, phạm vi nghiên cứu và phương pháp triển khai. Đề tài được lựa chọn từ danh sách do giảng viên cung cấp nhằm vận dụng các kiến thức đã học trong học phần Cấu trúc dữ liệu và giải thuật vào giải quyết một bài toán quản lý dữ liệu cụ thể.

Thông qua việc xác định các thuộc tính của sản phẩm như mã sản phẩm, tên sản phẩm, đơn giá, số lượng và loại sản phẩm, báo cáo đã làm rõ yêu cầu xây dựng chương trình quản lý bằng ngôn ngữ C++. Trong đó, mã sản phẩm được sử dụng làm khóa để tổ chức dữ liệu và hỗ trợ các thao tác tìm kiếm, thêm, xóa, cập nhật thông tin. Đây là cơ sở để lựa chọn cấu trúc dữ liệu phù hợp và xác định các chức năng cần triển khai.

Những nội dung đã trình bày trong chương này là nền tảng cho việc nghiên cứu cơ sở lý thuyết về cây nhị phân tìm kiếm ở chương 2, đồng thời định hướng quá trình phân tích và cài đặt chương trình trong các chương tiếp theo.

# CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

## 2.1. Khái niệm cây và cây nhị phân

Cây là cấu trúc dữ liệu phi tuyến gồm các nút được kết nối theo quan hệ cha – con. Nút đứng đầu được gọi là nút gốc. Một nút có thể có các nút con; nút không có nút con được gọi là nút lá. Khác với danh sách tuyến tính, dữ liệu trên cây có thể phân nhánh, phù hợp với các bài toán tìm kiếm và phân cấp.

Cây nhị phân là cây mà mỗi nút có tối đa hai nút con, thường gọi là cây con trái và cây con phải. Trong chương trình, mỗi Node có hai con trỏ left và right. Nếu một con trỏ bằng NULL, điều đó biểu thị nhánh tương ứng không có nút.

## 2.2. Cây tìm kiếm nhị phân (BST)

BST là cây nhị phân có thêm điều kiện thứ tự khóa. Với mỗi nút có khóa k, mọi khóa trong cây con trái nhỏ hơn k và mọi khóa trong cây con phải lớn hơn k. Quy tắc này phải đúng tại từng nút trong cây.

Trong chương trình, khóa là maSP có kiểu string. Vì vậy phép so sánh sử dụng thứ tự từ điển của chuỗi, không tự động hiểu mã là số.

Mã nguồn xử lý trường hợp trùng mã bằng cách tìm trước khi thêm. Hàm them() cũng chỉ đi sang trái khi mã nhỏ hơn và sang phải khi mã lớn hơn; nếu mã bằng nhau, hàm không tạo thêm nút.

## 2.3. Cấu trúc dữ liệu sử dụng

| Thành phần | Kiểu dữ liệu | Vai trò |
|---|---|---|
| maSP | string | Khóa định danh sản phẩm trong BST. |
| tenSP | string | Tên sản phẩm; nhập bằng getline để có thể chứa khoảng trắng. |
| donGia | double | Đơn giá sản phẩm. |
| soLuong | int | Số lượng sản phẩm trong kho. |
| loaiSP | string | Nhóm hoặc loại sản phẩm. |
| data | SanPham | Lưu toàn bộ thông tin sản phẩm trong một nút. |
| left, right | Node* | Con trỏ đến cây con trái và cây con phải. |
| root | Node* | Con trỏ đến nút gốc; NULL biểu thị cây rỗng. |

Cấu trúc SanPham gom các thuộc tính liên quan vào một đối tượng. Cấu trúc Node chứa đối tượng data và hai liên kết đến nút con. Hàm taoNode() cấp phát một nút mới bằng new, gán dữ liệu sản phẩm và khởi tạo hai con trỏ con bằng NULL.

### 2.4. Các phép duyệt cây

| Tên duyệt | Thứ tự thực hiện | Ứng dụng |
|---|---|---|
| NLR – tiền tự | Nút → Trái → Phải | In nút hiện tại trước khi xử lý các cây con; cũng là thứ tự được dùng khi ghi tệp. |
| LNR – trung tự | Trái → Nút → Phải | Với BST, cho ra các mã theo thứ tự tăng dần theo quy tắc so sánh khóa. |
| LRN – hậu tự | Trái → Phải → Nút | Phù hợp khi cần xử lý/xóa cây con trước nút cha. |

Các hàm duyetNLR(), duyetLNR() và duyetLRN() được cài đặt đệ quy. Mỗi hàm kiểm tra root == NULL để dừng nhánh. Nếu không có điều kiện dừng này, hàm có thể tiếp tục gọi xuống nhánh rỗng và gây lỗi.

### 2.5. Thuật toán thêm và tìm kiếm

Tìm kiếm bắt đầu từ gốc. Nếu cây rỗng, kết quả không tồn tại. Nếu khóa cần tìm bằng khóa tại nút hiện tại, hàm trả về nút đó. Nếu nhỏ hơn, thuật toán tìm ở cây con trái; ngược lại tìm ở cây con phải. Mỗi lần so sánh loại bỏ một nhánh khỏi phạm vi cần xét.

Thêm mới cũng dựa trên phép so sánh khóa. Khi đến vị trí NULL, chương trình tạo nút tại đó. Do nút được thêm vào vị trí lá thích hợp, tính chất BST được duy trì, với điều kiện không chèn khóa trùng.

### 2.6. Thuật toán xóa nút

Xóa là thao tác cần xử lý nhiều trường hợp vì việc loại bỏ một nút có thể ảnh hưởng đến liên kết của cây. Nút không có con trái: thay nút cần xóa bằng cây con phải, sau đó giải phóng nút cũ. Nút không có con phải: thay nút cần xóa bằng cây con trái, sau đó giải phóng nút cũ.

Nút có cả hai con: tìm nút nhỏ nhất trong cây con phải, sao chép dữ liệu nút đó vào nút cần xóa, rồi xóa nút kế nhiệm ở cây con phải. Cách xử lý trường hợp có hai con giúp duy trì quy tắc thứ tự BST. Hàm xoa() trả về con trỏ gốc của cây con sau khi xóa, vì vậy lời gọi ở main cần gán lại root = xoa(root, ma).

## 2.7. Độ phức tạp thuật toán

Gọi n là số nút trong cây và h là chiều cao cây. Các thao tác tìm kiếm, thêm và xóa phụ thuộc vào chiều cao h, do chỉ đi qua một đường từ gốc xuống một nhánh.

| Thao tác | Độ phức tạp theo chiều cao | Trường hợp cây cân đối tương đối | Trường hợp cây lệch |
|---|---|---|---|
| Tìm kiếm theo mã | O(h) | O(log n) | O(n) |
| Thêm sản phẩm | O(h) | O(log n) | O(n) |
| Xóa sản phẩm | O(h) | O(log n) | O(n) |
| Tìm mã nhỏ nhất/lớn nhất | O(h) | O(log n) | O(n) |
| Duyệt toàn cây / thống kê | O(n) | O(n) | O(n) |
| Hiển thị theo tầng | O(n) | O(n) | O(n) |

BST thông thường không tự cân bằng. Nếu lần lượt chèn các khóa đã tăng dần, mỗi nút mới có thể trở thành con phải của nút trước, khiến cây gần giống danh sách liên kết. Khi đó h có thể bằng n và lợi ích tìm kiếm giảm. Cây AVL hoặc Red-Black Tree là các hướng nâng cấp để kiểm soát chiều cao.

# KẾT CHƯƠNG 2

Chương 2 đã trình bày những kiến thức lý thuyết cần thiết phục vụ việc xây dựng chương trình quản lý sản phẩm bằng cây tìm kiếm nhị phân. Các nội dung chính bao gồm khái niệm cây, cây nhị phân, đặc điểm của cây tìm kiếm nhị phân, cấu trúc dữ liệu sử dụng trong chương trình và các phương pháp duyệt cây NLR, LNR, LRN.

Bên cạnh đó, chương đã phân tích nguyên lý của các thuật toán thêm, tìm kiếm và xóa nút trong BST, đồng thời xem xét độ phức tạp của các thao tác. Qua đó, có thể thấy hiệu quả của các thao tác tìm kiếm, thêm và xóa phụ thuộc vào chiều cao của cây. Khi cây tương đối cân đối, các thao tác này có thể đạt độ phức tạp trung bình O(log n); ngược lại, khi cây bị lệch, độ phức tạp có thể tăng lên O(n).

Những kiến thức trên là cơ sở quan trọng để lựa chọn cách tổ chức dữ liệu, xây dựng thuật toán và đánh giá hiệu quả hoạt động của chương trình. Trên nền tảng đó, chương 3 sẽ tập trung phân tích bài toán, xác định yêu cầu chức năng và thiết kế luồng xử lý tổng quát của hệ thống.

# CHƯƠNG 3. PHÂN TÍCH CHƯƠNG TRÌNH

## 3.1. Bài toán và dữ liệu đầu vào/đầu ra

Bài toán yêu cầu quản lý danh sách sản phẩm bằng BST, trong đó mã sản phẩm là khóa duy nhất. Dữ liệu đầu vào đến từ thao tác nhập của người dùng hoặc từ tệp sanpham.txt. Dữ liệu đầu ra gồm thông tin sản phẩm hiển thị trên màn hình, kết quả thống kê và nội dung được ghi ra tệp.

| Nhóm | Mô tả |
|---|---|
| Đầu vào | Lựa chọn menu; mã sản phẩm; tên; đơn giá; số lượng; loại; tên loại cần thống kê; thao tác đọc/lưu tệp. |
| Xử lý | Tìm kiếm và so sánh mã; chèn/xóa nút; cập nhật trường dữ liệu; duyệt cây; tính toán thống kê. |
| Đầu ra | Thông báo thành công/thất bại; thông tin sản phẩm; thứ tự duyệt; mã nhỏ nhất/lớn nhất; tổng giá trị kho; số lượng thống kê; nội dung tệp TXT. |

## 3.2. Yêu cầu chức năng

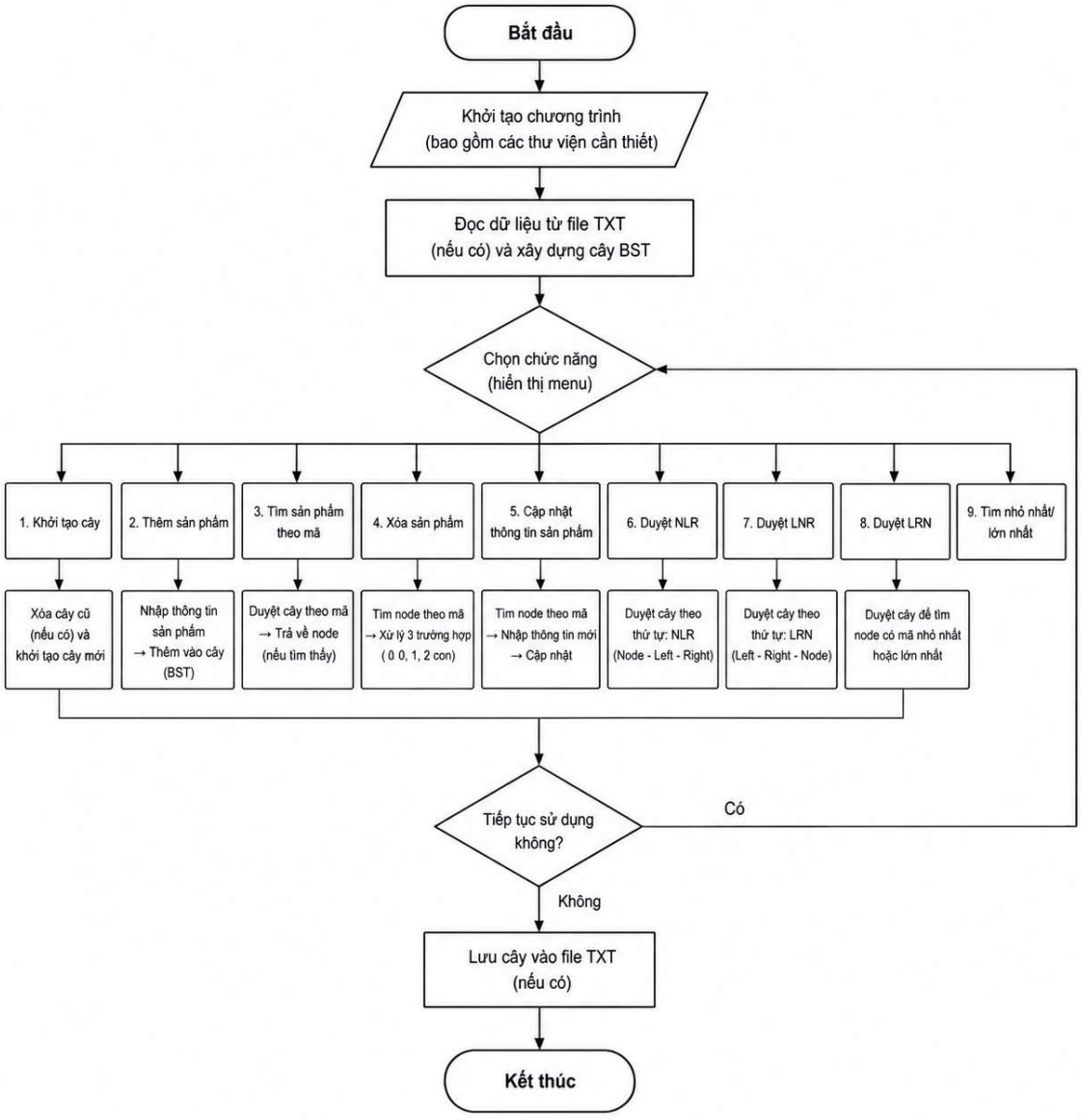
| STT | Chức năng | Mô tả xử lý |
|---|---|---|
| 1 | Khởi tạo cây | Ban đầu gán root = NULL; lựa chọn 1 trên menu cho phép nhập và thêm sản phẩm đầu tiên hoặc thêm tiếp vào cây. |
| 2 | Thêm sản phẩm | Nhập đủ năm thuộc tính, kiểm tra mã trùng rồi chèn theo quy tắc BST. |
| 3 | Tìm theo mã | Trả về thông tin sản phẩm nếu tìm thấy; nếu không, hiển thị thông báo. |
| 4 | Xóa sản phẩm | Tìm nút theo mã và xóa theo một trong ba trường hợp cấu trúc nút. |
| 5 | Cập nhật | Tìm nút và nhập lại tên, đơn giá, số lượng, loại; giữ nguyên mã khóa. |
| 6 | Duyệt NLR | In nút hiện tại trước, sau đó duyệt cây trái và cây phải. |
| 7 | Duyệt LNR | Duyệt cây trái, in nút, duyệt cây phải; mã được hiển thị tăng dần theo phép so sánh chuỗi. |
| 8 | Duyệt LRN | Duyệt hai cây con trước rồi mới in nút hiện tại. |
| 9 | Mã nhỏ nhất/lớn nhất | Đi về nhánh trái nhất hoặc nhánh phải nhất. |

### 3.3. Thiết kế cấu trúc và liên kết

Cấu trúc SanPham đóng vai trò bản ghi dữ liệu. Node kết hợp bản ghi với hai con trỏ. Gốc root là điểm bắt đầu của mọi thao tác. Khi cây chưa có sản phẩm, root bằng NULL. Sau lần thêm đầu tiên, root trỏ đến nút vừa tạo. Các thao tác có khả năng thay đổi cấu trúc như them() và xoa() trả về con trỏ gốc cây con để liên kết lại đúng.

Có thể mô tả quan hệ cấu trúc như sau: root trỏ đến Node; mỗi Node chứa data kiểu SanPham và hai liên kết left, right; mỗi liên kết có thể trỏ đến một Node khác hoặc NULL. Mã sản phẩm trong data là thuộc tính được dùng để quyết định hướng rẽ trái hay phải.

### 3.5. Sơ đồ luồng xử lý tổng quát




Hình 1. Sơ đồ luồng tổng quát

Vòng lặp do–while trong hàm main() bảo đảm menu được hiển thị lại sau khi hoàn thành một thao tác. Khi nhập không phải số nguyên, chương trình xóa trạng thái lỗi của cin, bỏ phần dữ liệu còn lại trên dòng và yêu cầu nhập lại. Khi lựa chọn bằng 0, vòng lặp kết thúc, giaiPhongCay() được gọi để giải phóng bộ nhớ.

### 3.6. Thiết kế menu

Menu chính có các lựa chọn từ 1 đến 18 và lựa chọn 0 để thoát. Hai lựa chọn 1 và 2 cùng dẫn đến quy trình nhập và thêm sản phẩm. Lựa chọn 10 mở menu phụ gồm lưu tệp và đọc tệp. Các lựa chọn còn lại gọi trực tiếp hàm tương ứng. Cách tổ chức này phù hợp với ứng dụng console và giúp người dùng thực hiện từng nghiệp vụ độc lập.

### 3.7. Quy tắc và giả định dữ liệu

- Mã sản phẩm không được trùng trong cây; so sánh mã theo thứ tự chuỗi.
- Tên và loại sản phẩm được nhập bằng getline nên có thể chứa dấu cách.
- Đơn giá được lưu bằng double và số lượng bằng int.
- Chương trình hiện chưa chặn đơn giá âm, số lượng âm hoặc giá trị nhập không hợp lệ cho các trường số.

# KẾT CHƯƠNG 3

Trong chương 3, báo cáo đã phân tích bài toán quản lý sản phẩm bằng cây tìm kiếm nhị phân, xác định dữ liệu đầu vào, quá trình xử lý và kết quả đầu ra của chương trình. Các yêu cầu chức năng cơ bản đã được làm rõ, bao gồm khởi tạo cây, thêm sản phẩm, tìm kiếm theo mã, xóa sản phẩm, cập nhật thông tin, duyệt cây theo ba thứ tự NLR, LNR, LRN và tìm sản phẩm có mã nhỏ nhất hoặc lớn nhất.

Chương cũng đã mô tả mối liên hệ giữa các thành phần dữ liệu SanPham, Node và con trỏ gốc root, qua đó làm rõ cách dữ liệu sản phẩm được tổ chức và liên kết trong cây. Sơ đồ luồng xử lý tổng quát và thiết kế menu giúp thể hiện trình tự thực hiện các chức năng, cách người dùng tương tác với chương trình và cách chương trình tiếp tục xử lý sau mỗi thao tác.

Ngoài ra, các quy tắc về mã sản phẩm không trùng lặp, cách so sánh mã theo thứ tự chuỗi và một số hạn chế trong kiểm tra dữ liệu đầu vào cũng đã được xác định. Đây là cơ sở để triển khai các hàm xử lý cụ thể trong chương 4, bảo đảm quá trình cài đặt bám sát yêu cầu và cấu trúc đã phân tích.

# CHƯƠNG 4. CÀI ĐẶT CÁC CHỨC NĂNG

## 4.1. Khởi tạo cây và tạo nút

Trong main(), cây được khởi tạo bằng câu lệnh Node* root = NULL. Điều này có nghĩa là chưa có nút gốc và cây chưa chứa sản phẩm nào. Hàm taoNode(SanPham sp) cấp phát vùng nhớ cho một Node mới, sao chép dữ liệu sản phẩm vào data, gán left và right bằng NULL rồi trả về địa chỉ nút.

Lựa chọn 1 được mô tả trên menu là khởi tạo cây, nhưng về cách cài đặt, chương trình thực hiện nhập một sản phẩm và thêm vào cây hiện tại, không xóa toàn bộ cây cũ. Vì vậy, nếu đã có dữ liệu, lựa chọn này tiếp tục thêm sản phẩm chứ không khởi tạo lại cây từ đầu.

## 4.2. Nhập và in thông tin sản phẩm

Hàm nhapSanPham() lần lượt nhận mã, tên, đơn giá, số lượng và loại sản phẩm. Sau khi đọc mã bằng cin, chương trình gọi cin.ignore() để bỏ ký tự xuống dòng còn lại trước khi dùng getline() đọc tên. Cách xử lý tương tự được áp dụng trước khi nhập loại sản phẩm.

Hàm inSanPham() hiển thị tất cả trường trên một dòng, gồm mã, tên, giá, số lượng và loại. Đây là hàm dùng chung trong các chức năng tìm kiếm, duyệt cây và tìm giá trị cực đại, giúp định dạng thông tin nhất quán.

## 4.3. Thêm sản phẩm

Hàm them() nhận gốc cây và đối tượng SanPham. Nếu gốc cây con bằng NULL, hàm tạo nút mới. Nếu mã nhỏ hơn khóa hiện tại, hàm đệ quy thêm vào cây con trái; nếu lớn hơn, thêm vào cây con phải. Nếu bằng nhau, hàm không chèn thêm nút.

Trong main(), chương trình gọi timTheoMa() trước khi thêm để thông báo rõ trường hợp mã đã tồn tại. Khi không trùng mã, root được gán bằng kết quả của them(root, sp). Việc gán lại root cần thiết vì nếu cây đang rỗng, nút mới sẽ trở thành gốc.

## 4.4. Tìm sản phẩm theo mã

Hàm timTheoMa() có ba kết quả có thể xảy ra tại mỗi bước: trả về nút nếu mã khớp, tiếp tục tìm ở trái nếu mã nhỏ hơn, hoặc tìm ở phải nếu mã lớn hơn. Khi đến NULL, hàm trả về NULL để báo không tìm thấy. Trong menu, chương trình dùng kết quả này để in thông tin hoặc thông báo sản phẩm không tồn tại.

## 4.5. Xóa sản phẩm

Trước khi xóa, main() kiểm tra mã có tồn tại hay không. Nếu có, gọi root = xoa(root, ma). Hàm xoa() tìm đúng nút theo quy tắc BST rồi xử lý theo số con của nút. Cách làm trên bảo đảm không cần di chuyển toàn bộ các nút trong cây. Tuy

nhiên, việc dùng con trỏ động đòi hỏi giải phóng đúng nút đã bị loại bỏ để tránh rò rỉ bộ nhớ. Hàm xoa() gọi delete root tại các trường hợp nút bị xóa trực tiếp.

### 4.6. Cập nhật thông tin sản phẩm

Hàm capNhat() tìm nút thông qua timTheoMa(). Nếu không tìm thấy, hàm thông báo và kết thúc. Nếu tìm thấy, chương trình nhập lại tên, đơn giá, số lượng và loại sản phẩm.

Mã sản phẩm không được cập nhật, vì đây là khóa đang xác định vị trí nút trong BST. Nếu cho phép thay đổi mã, cần xóa nút theo mã cũ rồi chèn lại với mã mới để duy trì thứ tự cây.

### 4.7. Duyệt NLR, LNR và LRN

Ba hàm duyệt đều dùng đệ quy và kết thúc ngay khi root bằng NULL. NLR in dữ liệu trước khi gọi hai cây con. LNR xử lý cây trái, in dữ liệu rồi xử lý cây phải. LRN xử lý hai cây con rồi mới in dữ liệu hiện tại. Sự khác nhau nằm ở thứ tự gọi hàm và vị trí in dữ liệu.

Trong ứng dụng quản lý sản phẩm, LNR đặc biệt hữu ích khi cần liệt kê sản phẩm theo mã tăng dần. NLR và LRN có giá trị minh họa các phương pháp duyệt cây và có thể dùng trong những thao tác xử lý cấu trúc cây khác.

```
// ==================== DUYET NLR ====================
void duyetNLR(Node* root) {
    if (root == NULL)
        return;

    inSanPham(root->data);
    duyetNLR(root->left);
    duyetNLR(root->right);
}

// ==================== DUYET LNR ====================
void duyetLNR(Node* root) {
    if (root == NULL)
        return;

    duyetLNR(root->left);
    inSanPham(root->data);
    duyetLNR(root->right);
}

// ==================== DUYET LRN ====================
void duyetLRN(Node* root) {
    if (root == NULL)
        return;

    duyetLRN(root->left);
    duyetLRN(root->right);
    inSanPham(root->data);
}
```

Hình 2. Duyệt NLR, LNR và LRN

### 4.8. Tìm mã nhỏ nhất và lớn nhất

Hàm timNhoNhat() bắt đầu từ gốc và đi theo con trỏ left cho đến khi không còn con trái. Nút cuối cùng là nút có khóa nhỏ nhất. Tương tự, timLonNhat() liên tục đi theo right để tìm khóa lớn nhất.

Hai hàm trả về NULL nếu cây rỗng. Trong menu lựa chọn 9, cả hai kết quả được hiển thị.

### 4.9. Tìm sản phẩm có giá cao nhất

Hàm timGiaCaoNhat() duyệt toàn bộ cây. Tham số kq là con trỏ tham chiếu dùng để lưu nút tốt nhất hiện tại. Nếu kq chưa có giá trị hoặc đơn giá tại nút đang xét lớn hơn đơn giá của kq, hàm cập nhật kq. Sau đó hàm tiếp tục xét cây trái và cây phải.

Vì giá không phải khóa của BST, không thể chỉ đi một nhánh để tìm giá lớn nhất; cần xét toàn bộ n nút.

### 4.10. Tìm sản phẩm có số lượng lớn nhất

Hàm timSoLuongLonNhat() có cách xử lý tương tự tìm giá cao nhất nhưng so sánh trường soLuong.

Kết quả là một nút có số lượng lớn nhất trong cây. Nếu nhiều sản phẩm đồng hạng, hàm chỉ lưu một nút đạt cực đại đầu tiên theo thứ tự duyệt; chương trình chưa liệt kê tất cả sản phẩm đồng hạng.

# KẾT CHƯƠNG 4

Chương 4 đã trình bày cách cài đặt các chức năng của chương trình quản lý sản phẩm bằng cây tìm kiếm nhị phân trên ngôn ngữ C++. Nội dung tập trung vào việc khởi tạo cây, tạo nút, nhập và hiển thị thông tin sản phẩm, thêm sản phẩm, tìm kiếm theo mã, xóa sản phẩm, cập nhật dữ liệu, duyệt cây theo các thứ tự NLR, LNR, LRN và tìm sản phẩm có mã nhỏ nhất hoặc lớn nhất. Chương cũng đã trình bày nguyên lý cài đặt các chức năng tìm sản phẩm có đơn giá cao nhất và sản phẩm có số lượng lớn nhất.

Thông qua quá trình cài đặt, các kiến thức lý thuyết về BST đã được vận dụng vào chương trình thực tế. Việc sử dụng đệ quy giúp các thao tác trên cây được triển khai theo cấu trúc rõ ràng; đồng thời, cơ chế cấp phát và giải phóng bộ nhớ động hỗ trợ quản lý các nút trong quá trình thêm và xóa sản phẩm. Việc sử dụng mã sản phẩm làm khóa cũng giúp duy trì quy tắc tổ chức dữ liệu của cây tìm kiếm nhị phân.

Tuy nhiên, chương trình vẫn còn một số hạn chế như chưa kiểm tra đầy đủ các giá trị đầu vào không hợp lệ, chưa tự cân bằng cây và chưa có giao diện đồ họa. Những hạn chế này là cơ sở để xem xét, đánh giá chương trình và đề xuất các hướng cải tiến phù hợp trong phần tổng kết báo cáo.

# KẾT LUẬN

Đề tài "Quản lý sản phẩm bằng cây tìm kiếm nhị phân" đã vận dụng cấu trúc BST để lưu trữ và xử lý dữ liệu sản phẩm theo mã. Chương trình C++ có các chức năng quản lý cơ bản gồm thêm, tìm, xóa, cập nhật, duyệt cây, tìm mã nhỏ nhất/lớn nhất và đọc. Ngoài ra, các chức năng mở rộng giúp thống kê giá cao nhất, số lượng lớn nhất, tổng giá trị kho, số sản phẩm theo loại, chiều cao cây, số nút lá, mức nút và hiển thị theo tầng.

Qua việc phân tích mã nguồn, có thể thấy BST là ví dụ trực quan về cách cấu trúc dữ liệu hỗ trợ thuật toán. Các thao tác tìm kiếm, thêm và xóa phụ thuộc vào chiều cao cây; do đó, cây cân đối thường xử lý tốt hơn cây lệch. Chương trình cũng cho thấy vai trò của đệ quy, con trỏ và quản lý bộ nhớ động trong C++.
Mặc dù đã đáp ứng phạm vi bài tập cơ bản, chương trình vẫn cần được hoàn thiện về kiểm tra dữ liệu đầu vào, xử lý lỗi khi đọc tệp và duy trì hiệu năng khi số lượng sản phẩm lớn.

# TÀI LIỆU THAM KHẢO

1. Tài liệu bài giảng học phần Cấu trúc dữ liệu và giải thuật.